Guided Lab 02: ReplicaSets – Keeping Your Pods Alive

Welcome to ReplicaSets

In our last lab, we created a single Pod. But here's the problem: if that Pod crashes or is deleted… it's gone. You'd have to manually re-create it. That doesn't scale, and it's certainly not what we want in a production system.

This is where **ReplicaSets** come in.

> A **ReplicaSet** ensures that *N* copies of a Pod are always running — no more, no less.

If one Pod dies, the ReplicaSet spins up another to replace it. Think of it like Kubernetes' way of keeping your app alive, automatically.

Step 1: Set Up Your Working Directory

Let's keep our labs organized.

```
cd ~/k8s-labs
mkdir -p replicasets
cd replicasets
```

Part 1: Writing a ReplicaSet YAML

We'll create a ReplicaSet that runs **3 NGINX Pods** and automatically recovers them if any fail or are deleted.

Step 1: Create the file

```
touch nginx-replicaset.yaml
```

Then open it:

```
nano nginx-replicaset.yaml
```

Step 2: Add the following YAML

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-replicaset
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.21.1
          ports:
            - containerPort: 80
```

What's happening here?

- `replicas: 3` tells Kubernetes: "I want three copies of this Pod."
- `selector.matchLabels` defines *which Pods* this ReplicaSet is responsible for. It must match the labels in the Pod template.
- `template` defines what each Pod should look like — same as the one we created earlier.

Step 3: Apply the ReplicaSet

```
kubectl apply -f nginx-replicaset.yaml
```

Now check if your Pods are running:

```
kubectl get pods
```

You should see **three Pods** created by the ReplicaSet.

Part 2: Let's Test the Healing Power

Now let's simulate a failure. Delete one of the Pods:

```
kubectl delete pod <one-of-the-pod-names>
```

Then immediately run:

```
kubectl get pods
```

You'll see Kubernetes **automatically recreates** the deleted Pod to maintain 3 replicas. This is auto-healing in action — powered by the ReplicaSet.

Bonus: Scaling Up Manually

Want more Pods?

You can edit the YAML and change:

```
replicas: 3
```

to:

```
replicas: 5
```

Then reapply:

```
kubectl apply -f nginx-replicaset.yaml
kubectl get pods
```

Now you'll see **five NGINX Pods**, all managed by the same ReplicaSet.

Useful Commands Recap

```
kubectl get replicaset
kubectl describe replicaset nginx-replicaset
kubectl delete replicaset nginx-replicaset
```

To see which ReplicaSet owns your Pod:

```
kubectl get pods -o wide
```

Wrap-Up: What Did You Learn?

- **ReplicaSet** ensures a fixed number of Pod replicas are always running
- If a Pod is deleted or crashes, ReplicaSet **auto-heals** it
- You can **scale manually** by changing the `replicas` count

- ReplicaSets are good, but they don't support **rolling updates** or **rollback** — which leads us to Deployments

What's Next?

In the next lab, we'll explore **Deployments**, which sit on top of ReplicaSets and bring production-grade features like rolling updates and rollbacks.

They are the recommended way to manage your apps in Kubernetes today.